

Intégration Continue



Théo BOUTROUX

Terry BARILLON

Paul SUPLOT

Louis BOUCARD

Rémi JEGARD

I – Introduction	2
II – Objectifs de la Pipeline	2
III – Déclencheurs de la Pipeline	2
IV – Étapes de la Pipeline	3
A – Build	3
B – Linter (Flake8)	3
C – SonarCloud	3
D – Trivy	4
E – Package	4
V – Conclusion	5

I – Introduction

Dans le cadre du développement d'un projet d'ELT en Python, nous avons mis en place une pipeline CI/CD en utilisant GitHub Actions. Cette pipeline automatise plusieurs étapes critiques du cycle de vie du projet à savoir : le contrôle qualité, la sécurité, l'analyse statique du code, et la livraison continue.

L'objectif de cette documentation est d'expliquer le fonctionnement, la pertinence et les apports concrets de cette pipeline.

II – Objectifs de la Pipeline

Les différents jobs mis en place au sein de notre pipeline Github Actions ont plusieurs objectifs qui sont :

- Fiabiliser le code grâce à l'automatisation des tests de qualité
- Détecter les vulnérabilités à un stade précoce
- Faciliter le packaging et la livraison du projet
- Garantir la conformité aux standards de code Python

III – Déclencheurs de la Pipeline

On retrouve deux déclencheurs automatiques au sein de notre pipeline. En effet, celle-ci va se déclencher automatiquement lors d'un push ou d'une pull request vers la branche main. Cela permet de garantir que tout nouveau code est contrôlé avant d'être intégré à la branche principale.

IV – Étapes de la Pipeline

A – Build

L'étape build est essentielle car elle permet d'installer les dépendances nécessaires au projet en utilisant le fichier requirements.txt. Elle permet donc de préparer l'environnement pour les jobs suivants

```
- name: Install dependencies
  run: |
    python -m pip install --upgrade pip
    pip install -r requirements.txt
```

B – Linter (Flake8)

Le linter Flake8 analyse le code Python pour identifier des erreurs de style, des incohérences avec les conventions PEP 8, ainsi que des problèmes de complexité cyclomatique. Il détecte également les erreurs critiques, comme les exceptions non gérées ou les variables inutilisées, qui peuvent affecter la performance et la maintenabilité du code. En imposant des règles strictes de codage, Flake8 aide à maintenir un code propre, lisible et facilement modifiable, tout en facilitant la collaboration entre les développeurs.

```
- name: Run linter
  run: |
    flake8 . --count --select=E9,F63,F7,F82 --show-source --statistics
    flake8 . --count --exit-zero --max-complexity=10 --max-line-length=88 --statistics
```

En quoi cette étape est utile ? Cela permet de garder un code propre, lisible et maintenable.

C – SonarCloud

SonarCloud est un outil d'analyse statique qui évalue la qualité du code en fournissant des métriques sur la couverture des tests, la dette technique et la sécurité. Il identifie les bugs potentiels, les vulnérabilités et les mauvaises pratiques qui pourraient compromettre la stabilité du projet. Grâce à son analyse approfondie, SonarCloud aide à prioriser les actions correctives et à maintenir un codebase sécurisé et durable.

```
- name: SonarCloud Scan
  uses: SonarSource/sonarcloud-github-action@v2
  env: SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
```

Pertinence : Cela améliore la qualité globale du code, en allant au-delà de ce que Flake8 peut détecter.

D – Trivy

Trivy est un scanner de sécurité qui analyse les dépendances du projet ainsi que les fichiers à la recherche de vulnérabilités critiques. Il détecte les failles de sécurité connues dans les bibliothèques utilisées et signale les risques potentiels. En intégrant Trivy dans le pipeline, on garantit que le projet reste sécurisé en identifiant rapidement les vulnérabilités avant qu'elles ne deviennent un problème.

```
- name: Run Trivy on dependencies
  run: trivy fs --exit-code 1 --severity HIGH,CRITICAL .
```

Apport : Cela permet d'éviter de déployer une application avec des failles de sécurité connues.

E – Package

Le projet est empaqueté sous forme de fichier .whl (wheel) en utilisant setuptools, ce qui est la norme pour la distribution de modules Python. Ce format permet une installation rapide et simple des packages Python, car il contient déjà le code compilé et les métadonnées nécessaires. L'utilisation de .whl facilite le déploiement et la gestion des versions du module tout en garantissant une compatibilité avec différents environnements.

```
- name: Build Python package
  run: |
    python setup.py sdist bdist_wheel

- name: Upload package artifact
  uses: actions/upload-artifact@v3
  with:
    name: python-package
    path: dist/*.whl
```

Apport : Ce fichier `.whl` peut être utilisé pour déployer facilement l'application sur un autre environnement ou serveur CI.

V – Conclusion

La mise en place de cette pipeline CI/CD apporte des bénéfices considérables pour le développement du projet Python. En automatisant des tâches essentielles, elle contribue à améliorer la qualité, la sécurité et l'efficacité de chaque phase du développement. Voici les principaux avantages :

> Livraison rapide et fiable

L'intégration continue permet de tester, valider et déployer rapidement le code, garantissant une livraison plus fréquente et plus fiable. Chaque changement est testé automatiquement avant d'être fusionné, ce qui permet de réduire les risques de bugs et d'assurer que le code reste fonctionnel à tout moment.

> Détection des erreurs et vulnérabilités

L'utilisation d'outils comme **Flake8**, **SonarCloud** et **Trivy** permet de détecter les erreurs de codage, les vulnérabilités de sécurité et les mauvaises pratiques dès qu'elles apparaissent. Cela garantit une qualité de code élevée tout au long du développement et permet d'intervenir rapidement en cas de problème.

> Standardisation des bonnes pratiques

L'automatisation des vérifications de style avec **Flake8** et des analyses de qualité de code avec **SonarCloud** garantit que tous les contributeurs suivent les mêmes bonnes pratiques. Cela assure une meilleure lisibilité et maintenabilité du code, tout en réduisant le temps passé à corriger des incohérences de style.

Cette pipeline CI/CD offre une approche structurée pour améliorer la qualité, la sécurité et la productivité du projet. Elle permet :

- Une **détection précoce des erreurs et vulnérabilités**.
- Une **livraison plus rapide et plus fiable**.
- Une **amélioration continue de la qualité du code**.

En somme, cette approche permet de maintenir un développement fluide et sécurisé, tout en assurant une livraison continue de code de haute qualité